Note méthodologique

Preuve de concept — Segmentation sémantique Cityscapes

Projet 9 — Veille & Amélioration

1. Dataset retenu

1.1. Présentation

Cityscapes est notre dataset de référence pour la segmentation sémantique en conduite autonome : 5 000 images (2048×1024) prises dans 50 villes européennes, avec masques annotés pixel par pixel.

Split officiel : 2 975 train, 500 val, 1 525 test. Référencé dans un CSV fixe pour garantir la reproductibilité.

1.2. Les 8 classes

Les 32 sous-catégories sont regroupées en 8 classes : void, flat, construction, object, nature, sky, human, vehicle. Remapping via lookup table dans preprocessing.py.

1.3. Déséquilibre

Flat et construction occupent > 70% des pixels. Object ne représente que ~2%, human ~0.4%. Ce déséquilibre se traduit directement dans les scores : ces deux classes ont toujours les IoU les plus basses.

1.4. Prétraitement

Images normalisées [0,1], redimensionnées à la résolution cible (128, 256 ou 512). Masques en interpolation nearest. Augmentation Albumentations : flip, rotation, luminosité/contraste, bruit, flou.

2. Les concepts de l'algorithme récent

Ce projet prolonge le Projet 8 (baselines CNN) en testant des architectures récentes (< 5 ans). On couvre tout le spectre : du CNN modernisé au foundation model auto-supervisé.

2.1. La base : U-Net et le transfer learning (rappel du Projet 8)

U-Net = encodeur (compresse l'image pour capter le contexte) + décodeur (reconstruit le masque) + skip connections (récupèrent les détails). Le transfer learning réutilise un encodeur pré-entraîné sur ImageNet.

Au Projet 8, on a établi les baselines : U-Net scratch, U-Net + VGG16, U-Net + ResNet50. Ces runs servent de référence pour mesurer le gain apporté par les nouvelles architectures.

2.2. ConvNeXt Tiny — le CNN revisité (2022)

ConvNeXt reprend ResNet et lui applique les idées des Vision Transformers : convolutions depthwise 7×7, LayerNorm, GELU, patchify stem. Résultat : aussi bon que Swin Transformer, avec la simplicité d'un CNN.

On l'utilise comme encodeur U-Net. Disponible dans tf.keras.applications.

2.3. SegFormer — le Transformer dédié (2021)

Architecture pure Transformer. Encodeur MiT (attention locale multi-échelle) + tête All-MLP (fusion des features sans FPN ni décodeur complexe).

On a testé **MiT-B0** (~3.8M params, légère) et **MiT-B5** (~82M params, plus riche). Observation clé : B5 converge 2× plus vite (148 min vs 273 min) pour un meilleur score (val mIoU 0.7213 vs 0.7140). Le facteur limitant, ce n'est pas le modèle, c'est la résolution (256x256).

2.4. EoMT / DINOv2 — foundation model auto-supervisé

DINOv2 (Meta AI, 2023)

Foundation model de vision, entraîné de manière auto-supervisée sur 142M images sans aucun label. ViT-B/14, ~86M params. Les features produites sont si bonnes qu'il suffit d'une petite tête pour segmenter. La version étendue (DINOv3) = variantes EoMT sur HuggingFace.

EoMT (2024)

Démontre qu'un simple ViT pré-entraîné + tête minimaliste suffit à battre Mask2Former et OneFormer. On a testé deux variantes :

- **Tête dense (notebook 7) :** dernière couche du ViT uniquement.
- **Tête multi-échelle + Cosine (notebook 7d) :** couches 9–12 fusionnées par poids appris + CosineAnnealingWarmRestarts.

SAM (Meta AI, 2023) — pour situer le contexte

Foundation model de segmentation interactive (1 milliard de masques). Fait de la segmentation d'instance, pas sémantique. Trop lourd pour une 1080 Ti. Notre approche DINOv2/EoMT partage la philosophie (foundation model + tête légère) mais en segmentation sémantique.

3. La modélisation

3.1. Métrique et loss

Elément	Description
Métrique	val_mIoU (Mean IoU sur validation, sans augmentation). Référence pour comparer les runs.
Loss	Cross-Entropy + 0.5 × Dice. CE gère chaque pixel, Dice aide les classes rares. Validé au P8 : CE seule plafonne à 0.63–0.65, CE+Dice monte au-delà de 0.75.
Optimiseur	Adam (TF/Keras) ou AdamW avec LR différentiel (PyTorch/EoMT).
Seed	42 pour tous les runs.

3.2. Paramètres explorés

Au total, **27 runs** consignés dans comparison_all_runs.csv.

Paramètre	Valeurs testées & impact
Résolution	128×128, 256×256, 512×512. Levier n°1 : passer de 256 à 512 donne +3 à +6 pts.
Batch size	1, 2, 4, 8. Batch 4 = meilleur compromis sur 1080 Ti (11 Go VRAM).
Augmentation	Avec vs sans. Le scratch 256 passe de 0.6477 (sans) à 0.7539 (avec). +10 pts.
Loss	CE seule vs CE+Dice (λ=0.5). CE+Dice systématiquement meilleur.
Époques	5, 10, 20, 30, 40, 50+30 (2 phases). Plus = mieux, jusqu'au plateau.
Mode	scratch (aucun pré-entraînement), frozen (encodeur gelé), finetune (tout dégelé).
Modèles	Scratch, VGG16, ResNet50, ConvNeXt Tiny, SegFormer B0, SegFormer B5, EoMT DINOv2.

3.3. Protocole d'entraînement

Phase 1 — Encodeur gelé

On entraîne uniquement la tête. 50 époques max, LR 1e-3. La tête s'adapte aux features du backbone.

Phase 2 — Fine-tuning

On dégèle tout. 30 époques max, LR 1e-4. Pour EoMT : LR différentiel (encodeur = 0.1×LR tête).

Callbacks

ModelCheckpoint (sauvegarde le meilleur val_mIoU), EarlyStopping (patience 8–10), ReduceLROnPlateau (factor=0.5). EoMT notebook 7d : CosineAnnealingWarmRestarts (T0=10, mult=2).

3.4. Organisation du code

10 notebooks (1 par modèle/analyse) + scripts partagés (preprocessing, datagen, augmentations, losses, models, training, inference, viz, seed, config). Seul le modèle change entre notebooks.

Deux environnements : TF/Keras (.env0) pour CNN + SegFormer ; PyTorch (.env_eomt, torch 2.0.1+cu117) pour EoMT/DINOv2.

4. Synthèse des résultats

4.1. Projet 8 — les baselines (à 256×256)

Les runs du Projet 8, tous en 256×256, batch 4, avec augmentation et loss CE+Dice (sauf mention contraire) :

Modèle	Résol.	Ép.	Val mIoU	Mode	Loss
U-Net Scratch	256	40	0.7539	scratch	CE+Dice
U-Net + VGG16 FT	256	30	0.7465	finetune	CE+Dice
U-Net + ResNet50 FT	256	30	0.7471	finetune	CE+Dice
U-Net + ResNet50 Frozen	256	40	0.7440	frozen	CE+Dice
U-Net + VGG16 Frozen	256	40	0.7170	frozen	CE+Dice
Scratch (10 ep)	256	10	0.6593	scratch	CE+Dice
Scratch (sans aug, CE seule)	256	5	0.6477	scratch	CE+Dice
Scratch 128×128, b8, sans aug	128	5	0.6281	scratch	CE+Dice
Scratch 128×128, b8, aug, CE	128	10	0.6044	scratch	CE+Dice

Le meilleur run baseline est le **scratch à 40 époques (0.7539)**, talonné par ResNet50 FT (0.7471) et VGG16 FT (0.7465). Les runs en bas du tableau montrent l'impact de la résolution (128 vs 256), de l'augmentation et de la loss. Plus tard, les runs de 512 (vs 256 ici) montreront que leur mIoU augmenteront significativement et battront le scratch classique.

4.2. Projet 9 — Nouveaux modèles (à 256×256)

Les runs ajoutés au Projet 9, en gardant le même protocole que la baseline pour une comparaison équitable :

Modèle	Résol.	Ép.	Val mIoU	Mode	Loss
ConvNeXt Tiny FT (court)	256	~30	0.7738	finetune	CE+Dice
ConvNeXt Tiny FT (50+30)	256	50+30	0.7598	finetune	CE+Dice
ConvNeXt Tiny Frozen	256	~40	0.7575	frozen	CE+Dice
EoMT MultiScale + Cosine	256	50+30	0.7338	finetune	CE+Dice
SegFormer MiT-B5 (court)	256	10+5	0.7213	finetune	CE+Dice
SegFormer B0 FT (40 ep)	256	40	0.7211	finetune	CE+Dice
SegFormer B0 (50+30)	256	50+30	0.7140	finetune	CE+Dice
SegFormer B0 FT (30 ep)	256	30	0.7131	finetune	CE+Dice
EoMT DINOv2 tête dense	256	50+30	0.7119	finetune	CE+Dice

Que peut-on en tirer ?

ConvNeXt Tiny prend la tête. Son meilleur run à 256 (val mIoU 0.7738) dépasse la meilleure baseline du P8 (scratch à 0.7539). Même en frozen, il fait 0.7575.

EoMT multi-échelle à 0.7338, honorable pour un modèle auto-supervisé avec une tête légère. Converge en seulement 168 min.

MiT-B5 vs B0 : B5 atteint 0.7213 en 15 époques (148 min), là où B0 met 80 époques (273 min) pour 0.7140. Plus gros = plus rapide, car les features sont plus riches.

4.3. Projet 9 — Quickbench 512×512

Tous les modèles ont été benchés à 512×512 (20 époques, batch 4, augmentation) :

Modèle	Résol.	Ép.	Val mIoU	Mode	Loss
ConvNeXt Tiny FT	512	20	0.8256	finetune	CE+Dice
VGG16 FT	512	20	0.7970	finetune	CE+Dice
ConvNeXt Tiny Frozen	512	20	0.7965	frozen	CE+Dice
SegFormer B0 Frozen	512	20	0.7805	frozen	CE+Dice
SegFormer B0 FT	512	20	0.7782	finetune	CE+Dice
ResNet50 Frozen	512	20	0.7754	frozen	CE+Dice
ResNet50 FT	512	20	0.7715	finetune	CE+Dice
Scratch	512	20	0.7365	scratch	CE+Dice
VGG16 Frozen	512	20	0.7290	frozen	CE+Dice

L'impact de la résolution, en chiffres

Modèle	Val 256	Val 512	Gain	Remarque
ConvNeXt FT	0.7738	0.8256	+5.2 pts	N°1 global
SegFormer B0 Frozen	~0.71*	0.7805	+6.8 pts	Gros gain
VGG16 FT	0.7465	0.7970	+5.1 pts	VGG vieillit bien
Scratch	0.7539	0.7365	−1.7 pts	Sous-entraîné (20ep)

Le passage à 512 est le gain le plus facile à obtenir. Le scratch 512 semble reculer, mais c'est un artefact : il n'a eu que 20 époques là où le run 256 en avait 40.

4.4. Les classes en difficulté : object et human

Quel que soit le modèle, le pattern est le même :

- Flat (IoU > 0.95), sky (> 0.88), vehicle (> 0.82) : grandes régions homogènes, bien segmentées.
- Object (0.30–0.39), human (0.57–0.63) : petits, rares, géométrie variable, toujours en retrait.

EoMT DINOv2 obtient les meilleures IoU sur ces deux classes (object 0.392, human 0.627). Les features auto-supervisées semblent mieux capter les petits objets.

Pistes pour améliorer ces classes

- **Class weights :** surpondérer human et object dans la loss pour forcer le modèle à mieux les discriminer.
- **Focal Loss :** se concentrer sur les pixels les plus difficiles à classifier.
- **Résolution :** monter en 512 (voir plus) améliore la segmentation des petits objets.

5. Analyse de la feature importance

5.1. Au niveau global

Skip connections (U-Net). Features basse résolution = contexte sémantique. Haute résolution = détails spatiaux. Indispensables.

Fusion multi-échelle (EoMT 7d). Les poids appris révèlent que les couches 12 (globale) et 9 (textures) contribuent le plus.

Qualité des features DINOv2. Encodeur gelé + tête légère (~1M params) = val_mIoU > 0.63 dès la phase 1. Ça montre la puissance du pré-entraînement auto-supervisé.

5.2. Au niveau local (par classe)

Les confusions principales (matrice de confusion) : human/vehicle aux bordures, object/construction pour les poteaux. Les visualisations RGB/GT/Pred de chaque notebook confirment que EoMT est plus précis sur les petits objets, et ConvNeXt sur les grandes surfaces.

L'augmentation joue aussi un rôle de régularisation : sans elle, le modèle surapprend sur les classes majoritaires et ignore les rares.

6. Dashboard et déploiement

6.1. Contexte : le Projet 8 avait une API + app de prédiction

Au Projet 8, on avait mis en production une **API FastAPI** (4 endpoints : health, predict/mask, predict/mask_color, predict/overlay) et une **app Streamlit de prédiction** qui uploadait une image, appelait l'API et affichait le masque colorisé + overlay + stats par classe. Le modèle était stocké sur HuggingFace et téléchargé au démarrage. Ce système reste fonctionnel mais n'est pas au cœur du Projet 9.

6.2. Projet 9 : le dashboard de comparaison

Le livrable principal du Projet 9 est un **dashboard Streamlit de comparaison**, déployé sur Replit, qui permet de naviguer dans l'ensemble des résultats du benchmark.

- **Page : EDA**

Présentation du dataset : exemples d'images + masques par split et par ville, comptage des pixels par classe (histogrammes interactifs), visualisation des transformations (equalisation, flou, augmentations Albumentations). Fonctionne en mode cache (déployé sans les 10 Go du dataset) ou en mode live (local).

- **Page : Comparaison**

C'est la page centrale du dashboard. Elle charge le CSV comparison_all_runs.csv, et les fichiers de chaque run dans out/experiments/ contenant l'historique des runs et les data sous .json, ainsi que les graph d'IoU et de Loss sous .png ; et permet de :

- Filtrer et trier les 27 runs par encodeur, résolution, mode, score.
- Afficher un bar chart des top N runs, un scatter encodeur vs val_mIoU, un boxplot par groupe.
- Tracer les courbes de loss et de mIoU de n'importe quel run (graphiques Plotly interactifs).
- Voir les images de synthèse de chaque run (grille de prédictions, courbes loss/mIoU).

- **Page : À propos**

Récapitulatif des modèles, de la méthodologie, de la palette des 8 classes, et des critères d'accessibilité.

6.3. Accessibilité WCAG

Un toggle « Mode accessible » active : police 18px, hachures sur les barres (1.4.1), bordures sur les scatter (1.4.11), palette haut contraste (1.4.3), focus clavier orange (2.4.7), descriptions textuelles (1.1.1), espacement conforme (1.4.12).

6.4. Déploiement

Le dashboard est déployé sur **Replit** via intégration GitHub. Le script precompute_eda.py précalcule les stats du dataset dans un cache (eda_cache/). Un pipeline CI/CD (flake8 + 12 tests pytest) assure la qualité du code. Les modèles lourds (.keras, .pt) restent au besoin sur HuggingFace.

7. Limites et améliorations possibles

7.1. Limites

Résolution. Le principal entraînement est à 256. On sait que 512 donne +3 à +6 pts en IoU, mais les runs 512 n'ont eu que 20 epochs.

Pas de class weights. Les classes rares (object, human) mériteraient une surpondération.

Pas de TTA. Pas de flip/multi-scale à l'inférence. Gain estimé : +1 à +2 pts.

GPU 1080 Ti (sm_61). Pas de mixed precision, pas de CUDA 12.x. , ça limite la résolution et le batch.

EoMT simplifié. On n'a pas implémenté les mask queries.

7.2. Améliorations

Class weights / Focal Loss pour booster object et human (et voir pour les autres class en difficultés).

Entraînement complet 512 et 1024 (40+30 epoch maximum) avec gradient accumulation. ConvNeXt devrait dépasser 0.85 en 512 et 0.90 en 1024.

MiT-B5 à 512 : B5 converge plus vite, à 512 il pourrait rivaliser avec ConvNeXt (voir 1024).

TTA : flips + multi-scale à l'inférence + autres paramètres.

CRF en post-processing pour affiner les contours.

EoMT complet avec mask queries injectées + autres paramètres.

Encodeurs plus lourds (ConvNeXt Small, Base … MiT-B3, DINOv2-Large, DINOv3), si mémoire GPU > 11 Go et CUDA > 11.x.x, avec bibliothèques plus récentes et modèles plus lourds.

Ce projet a permis de passer d'une baseline classique (U-Net + VGG16/ResNet50) à des architectures récentes qui améliorent concrètement les résultats.

ConvNeXt Tiny prend la tête du classement, que ce soit à 256 ou à 512. EoMT/DINOv2 prouve que les foundation models auto-supervisés ont leur place en segmentation, même sur un GPU modeste.

Le test de MiT-B5 révèle qu'un encodeur plus lourd peut accélérer la convergence plutôt que de la ralentir.

Il reste encore de la marge : les class weights, l'entraînement complet à 512 (et 1024) et le TTA sont les prochaines étapes logiques pour aller plus loin.